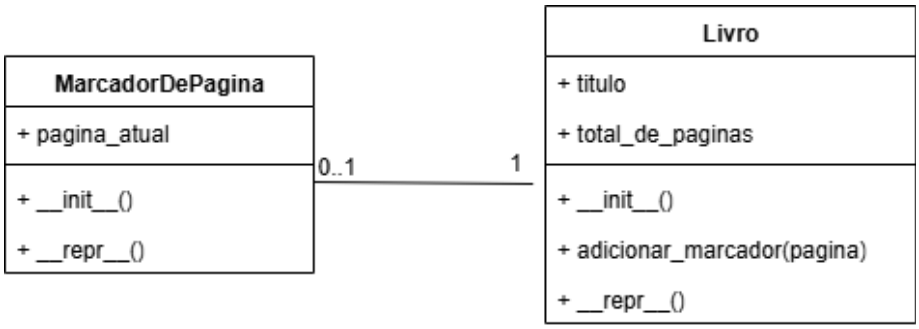
Curso Técnico Integrado em Informática
Unidade Curricular: Programação e Engenharia de Software
Professores: Thiago Waltrik e João Franken

Lista de Exercícios 07

1 - Livro e MarcadorDePagina (0..1).
Objetivo: Implementar um relacionamento opcional simples.
Classes: MarcadorDePagina e Livro.
Regras:

- A classe MarcadorDePagina deve ter um atributo pagina_atual.
- A classe Livro deve ter um atributo titulo e um total_paginas.
- Um Livro pode ter zero ou um MarcadorDePagina (ele começa sem marcador, 0..1).
- Crie um método no Livro chamado adicionar_marcador(pagina). Este método deve criar uma instância de MarcadorDePagina e associá-la ao livro. Se o livro já tiver um marcador, apenas atualize a pagina_atual dele.

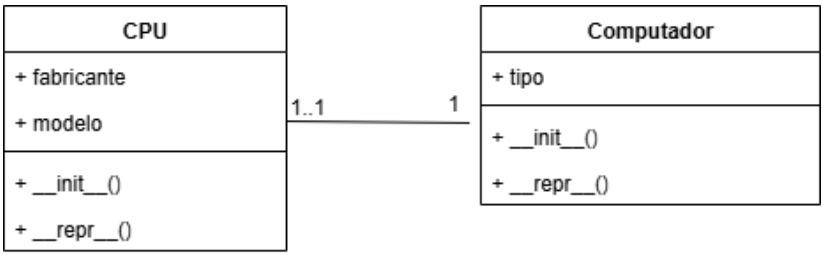
Comprove o funcionamento da sua implementação no programa principal.
Veja, na figura abaixo, o Diagrama de Classes completo:



2 - Exercício 2: Computador e CPU (1..1).
Objetivo: Implementar um relacionamento obrigatório (Composição).
Classes: CPU e Computador.
Regras:
A classe CPU deve ter atributos fabricante (ex: "Intel") e modelo (ex: "i7-12700K").

- A classe Computador deve ter um atributo tipo (ex: "Desktop" ou "Notebook").
- Um Computador deve ter exatamente uma CPU para ser criado (1..1).
- O __init__ do Computador deve receber os dados necessários para criar sua própria instância de CPU. O atributo computador.cpu nunca pode ser None.

Comprove o funcionamento da sua implementação no programa principal.
Veja, na figura abaixo, o Diagrama de Classes completo:



3 - VagaDeEstacionamento e Carro (0..1 - Associação Simples)
Objetivo: Modelar um relacionamento 0..1 onde a "parte" (Carro) vem de fora (Agregação/Associação).
Classes: Carro e VagaDeEstacionamento.
Regras:
A classe Carro é simples, tem apenas placa e modelo.

- A classe VagaDeEstacionamento tem um id_vaga (ex: "A-101").
- Uma VagaDeEstacionamento pode estar ocupada por zero ou um Carro (0..1). O atributo carro_estacionado começa como None.
- Crie um método ocupar_vaga(carro_obj). Este método deve verificar se a vaga já está None. Se estiver livre, associa o carro_obj (vindo de fora) à vaga e retorna True. Se já estiver ocupada, não faz nada e retorna False.
- Crie um método liberar_vaga() que simplesmente seta carro_estacionado de volta para None.

Seu programa principal deve ter um menu interativo onde se possa testar todos os métodos de um objeto do tipo VagaDeEstacionamento. Faça, também, o Diagrama de Classes da sua implementação final.

4 - Exercício: Funcionario e ContratoDeTrabalho (1..1).
Objetivo: Modelar um 1..1 onde a "parte" (Contrato) é obrigatória e contém a lógica de cálculo que o "todo" (Funcionario) utiliza.
Classes: ContratoDeTrabalho e Funcionario.
Regras:

- A classe ContratoDeTrabalho deve ter salario_bruto (float) e percentual_impostos (float, ex: 0.11 para 11% de INSS).
- O ContratoDeTrabalho deve ter um método chamado valor_impostos que calcula e retorna o valor exato do desconto (ex: salario_bruto * percentual_impostos).
- A classe Funcionario deve ter nome e cargo.
- Um Funcionario deve ter exatamente um ContratoDeTrabalho para ser criado (1..1).
- O __init__ do Funcionario deve receber nome, cargo, e também o salario_bruto e o percentual_impostos necessários para criar sua própria instância de ContratoDeTrabalho (Composição). O atributo funcionario.contrato nunca pode ser None.
- Crie um método chamado salario_liquido na classe Funcionario. Este método deve:
  - Acessar o salario_bruto do seu contrato.
  - Acessar o valor_impostos calculado pelo seu contrato.
  - Retornar o salario_bruto menos o valor_impostos.

Seu programa principal deve ter um menu interativo onde se possa testar todos os métodos de um objeto do tipo Funcionario. Faça, também, o Diagrama de Classes da sua implementação final.